smaller events, and publishes them for later consumption on a persistent event log, like a Kafka topic.

Another example is using the outbox pattern. In a nutshell, whenever the Messflix Streaming Platform saves its new state, it is also in the same transaction saving related events into a database table called the *outbox table*. Events are later extracted from the table by an event publisher (such as Kafka Connect or any other similar tool). The event publisher, in this case, will be our Content Interaction data product.

OK, we are able to publish events from our source data products, but what if we want to process data in a batch that requires files?

DATA LAKE AND DATA PRODUCTS

With batch processing, we can expose our data in the form of files to make calculations efficient. But what if we are already using some kind of data lake?

This is the right time to make an important point: a data mesh implementation does not exclude the possibility of using a data lake! In fact, both work together pretty well. What changes is the ownership of the data and data pipelines, and ownership of data lake technology: the platform.

KEY POINT A data mesh implementation does not exclude the possibility of using a data lake; these concepts can be complementary.

When we apply the data mesh way of thinking to a data lake, we see that data product teams will own their areas/folders/buckets/namespaces within the same data lake persistence layer where they store data related to their data product. They will also own data pipelines that transform this data, such as those for cleansing operations. Data lake as a platform is owned not by data product teams but by a platform team. Figure 9.22 shows an example of this setup.